

# Computer Architecture: CS-643

## Lab 2: Pipelining and Superscalar using SimpleScalar

October 23, 2025

### Objective

In this lab, you will use the SimpleScalar toolset to explore the concepts of pipelined and superscalar processor architectures. You will configure and simulate both in-order and out-of-order processor models to understand their impact on performance and to determine if features like data forwarding are implemented.

### Background: The **sim-outorder** Simulator

We will use **sim-outorder**, a detailed microarchitectural timing simulator from the SimpleScalar toolset. It models a modern superscalar processor with features like branch prediction, caches, and out-of-order execution. By adjusting its configuration parameters, we can simulate various processor designs and analyze their performance, measured in Instructions Per Cycle (IPC).

All required tools are located in the **simplesim-3.0/** directory relative to your project root. You can get help for any simulator by running it with the **-h** flag:

```
./simplesim-3.0/sim-outorder -h
```

---

### Part 1: In-Order Pipelined Processor

In this part, we will simulate a simple, in-order processor and analyze its behavior to check for data forwarding.

#### Procedure

1. **Create a Workspace:**

From your project root directory (**/home/palhad/SimpleScalar/**), create a new directory for this lab.

```
mkdir labs
```

2. **Copy Configuration Files:**

Copy the default simulator configuration file into your new **labs** directory.

```
cp simplesim-3.0/config/default.cfg labs/
```

3. **Prepare **config.a.cfg**:**

Make a copy of the configuration file for this part of the experiment.

```
cp labs/default.cfg labs/config_a.cfg
```

4. **Copy Benchmark Program:**

Copy the **test-math** pre-compiled binary into your **labs** directory.

```
cp simplesim-3.0/tests-pisa/bin.little/test-math labs/
```

### 5. Modify config\_a.cfg:

Open labs/config\_a.cfg in a text editor. Find and modify the following parameters to match the values below.

```
-decode:width 1 (Sets how many instructions can be decoded per clock cycle.)
-fetch:ifqsize 1 (Defines the size of the buffer holding instructions before
    ↳ they are decoded.)
-lsq:size 8 (Sets the number of memory operations (loads/stores) that can be
    ↳ active at once.)
-ruu:size 8 (Defines the size of the main instruction window, holding all "in-
    ↳ flight" instructions.)
-issue:width 1 (Sets how many instructions can be sent to the execution units
    ↳ per clock cycle.)
-issue:inorder true (Forces instructions to execute in program order (true) or
    ↳ allows out-of-order execution (false).)
-res:ialu 1 (The number of available integer arithmetic units (for add, sub,
    ↳ etc.).)
-res:imult 1 (The number of available units for integer multiplication and
    ↳ division.)
-res:memport 1 (The number of available ports to access the data cache/memory.)
-res:fpalu 1 (The number of available floating-point arithmetic units.)
-res:fpmult 1 (The number of available units for floating-point multiplication
    ↳ and division.)
```

### 6. Run the Simulation:

Navigate to your labs directory and run the simulator with the new configuration.

```
cd labs
../simplesim-3.0/sim-outorder -config config_a.cfg -ptrace config_a.trc 0:1024
    ↳ -redir:sim sim_configa.out ./test-math
```

### 7. Analyze the Results:

The simulation statistics are now in sim\_configa.out. The pipeline trace is in config\_a.trc.

## Questions for Part 1

1. Based on the settings in config\_a.cfg, describe the processor's architecture.
2. Open the sim\_configa.out file. What is the final value for sim\_IPC (Instructions Per Cycle)?
3. Is data forwarding implemented? To answer this, examine the pipeline trace for a RAW (Read-After-Write) hazard. Look for a sequence like:

```
ag: sll r2, r16, 2
ah: addu r3, r3, r2
```

Use the pipeview.pl script to view the trace. Can you find a cycle where the addu instruction is in the Execute (EX) stage while the sll instruction is in the Write-Back (WB) stage?

```
../simplesim-3.0/pipeview.pl config_a.trc | less
```

---

## Part 2: Out-of-Order Superscalar Processor

Now, we will modify the configuration to model an out-of-order processor and compare its performance.

## Procedure

### 1. Prepare **config.b.cfg**:

Make sure you are in your labs directory. Make a copy of `config_a.cfg`.

```
# If not already in the labs directory:  
# cd labs  
cp config_a.cfg config_b.cfg
```

### 2. Modify **config.b.cfg**:

Open `config_b.cfg` and apply the same modifications as in Part 1, with **one exception**:

```
-issue:inorder false
```

### 3. Run the Simulation:

Run the simulator with the `config_b.cfg` file.

```
../simplesim-3.0/sim-outorder -config config_b.cfg -redir:sim sim_configb.out  
→ ./test-math
```

## Questions for Part 2

1. Describe the new configuration. What is the key difference compared to the configuration in Part 1?
2. Open the `sim_configb.out` file. What is the `sim_IPC` for this configuration?
3. Compare the IPC from Part 1 and Part 2. What does this tell you about the benefit of out-of-order execution for this workload?
4. In an out-of-order processor, the Commit (CT) stage ensures results are written back in program order. Why is this in-order commit mechanism necessary?